

DEVELOPMENT PHASE ROADMAP

OMSKing

Leheriya Omnichannel OMS — Phase-by-Phase Cursor AI Build Plan

Total Phases	13 (Phase 0–12)
Total Modules Covered	23
Basis	Final Combined Agreement & Scope
Purpose	Prevent Cursor AI context loss, wrong architecture, and messy code

Why 13 Phases

Splitting the project into too many small phases (e.g. 15+) wastes Cursor AI's context on trivial handoffs. Combining too much into too few phases (e.g. 10, with heavy modules like Shipping and Returns merged together) risks Cursor losing context mid-phase, producing wrong architecture, or generating messy, tangled code. The 13-phase plan below groups only what is genuinely dependent on the same data or logic, and keeps every critical or API-heavy module — like Inventory and Shipping — as its own isolated phase.

Rule to give Cursor AI before every phase:

Before every phase, give Cursor AI this instruction: “We are working only on PHASE X right now. Do not modify any other module. First analyze the existing codebase, then propose an implementation plan, and only after the plan is approved, implement the code for this approved scope.”

Workflow to repeat for every phase:

- 1 Explain the project and this phase's scope to Cursor
- 2 Have Cursor analyze the existing codebase
- 3 Have Cursor produce an implementation plan
- 4 Review and approve the plan
- 5 Implement only the approved scope
- 6 Test thoroughly
- 7 Git commit
- 8 Move to the next phase

Phase Overview

#	Phase	Core Modules
0	Architecture & Foundation	Planning only — no application code written yet
1	UI Foundation & Design System	Dummy-data skeleton for every module — no backend connected yet
2	Authentication + RBAC + Multi-Tenant	Modules: Authentication, Tenant/Merchant Mgmt, Users & Roles
3	Products + Master SKU + SKU Mapping	The first real business-data core module
4	Warehouses + Inventory Engine	The single most critical phase in the entire project
5	Orders + Fulfilment	Modules: Orders, Fulfilment
6	Vendor Management	Module: Vendors
7	Shipping	Module: Shipping — kept as its own phase due to third-party API complexity
8	Returns + RTO + NDR	Modules: Returns, RTO, NDR — combined because they share the same workflow pattern
9	GST Invoice + Payment & Return Reconciliation	Modules: GST Invoice, Payment Reconciliation, Returns/Refunds Reconciliation
10	Notifications + Settings + Integration Management	Modules: Notifications, Settings, Integration Management
11	Dashboard + Reports + Audit Logs	Modules: Dashboard, Reports, Audit Logs
12	Testing, Optimization & Deployment	Final phase before go-live

Detailed Phase Breakdown

PHASE 0 — Architecture & Foundation

Planning only — no application code written yet

Goal

Lock the technical blueprint so every later phase builds on a stable, agreed foundation.

Tasks

- 1 Finalize tech stack: React.js (Admin + Vendor panel), Node.js (Express/NestJS) backend, PostgreSQL database
- 2 Draft full database schema — Tenants, Users, Roles, Products, Master_SKU, SKU_Mapping, Warehouses, Inventory_Ledger, Orders, Order_Items, Vendors, Shipments, Returns, Audit_Logs
- 3 Decide multi-tenant strategy — shared database with tenant_id row-level isolation (simplest, scalable for SaaS)
- 4 Lock folder structure — separate admin panel, vendor panel, backend API, shared utilities
- 5 Define REST API conventions — versioning (/api/v1/...), standard error response format
- 6 Create GitHub repository and share access with client (per agreement, from Day 1)
- 7 Write coding standards document — naming conventions, commit message format, branch strategy

Output: Architecture document + ER diagram + empty repo with folder skeleton

PHASE 1 — UI Foundation & Design System

Dummy-data skeleton for every module — no backend connected yet

Goal

Get the entire application visually walkable before any real logic is built, so UX issues are caught early and cheap.

Tasks

- 1 Build Admin layout — sidebar, header, responsive shell
- 2 Build Vendor layout — deliberately simpler, Excel-style, per the agreement's design principle
- 3 Build shared design system — buttons, form fields, tables, modals, loaders, empty/error states
- 4 Create skeleton pages (dummy data) for all core modules: Dashboard, Orders, Products, Inventory, Warehouses, Vendors, Shipping, Returns, Settings, Reports

Output: Fully clickable UI shell across admin + vendor panels

■ No API integration in this phase — dummy data only.

PHASE 2 — Authentication + RBAC + Multi-Tenant

Modules: Authentication, Tenant/Merchant Mgmt, Users & Roles

Goal

These three are tightly coupled at the database level — splitting them risks an inconsistent permission model, so they are built together.

Tasks

- 1 Login / Logout API with JWT + Refresh Token
- 2 Password reset flow
- 3 Tenant table + tenant-identification middleware on every request
- 4 Roles table — Super Admin (full access) and Vendor (restricted) at launch
- 5 Permission middleware — route-level and action-level access control
- 6 Session management and token expiry handling

Output: Working login system with tenant isolation and role-based route protection

PHASE 3 — Products + Master SKU + SKU Mapping

The first real business-data core module

Goal

Establish the Master SKU as the single identity every other module (inventory, orders, vendors, shipping) will key off.

Tasks

- 1 Product CRUD — variants, images, pricing, GST/HSN fields
- 2 Master SKU generation logic — one unique internal SKU per sellable variant
- 3 Channel SKU mapping table — Shopify / Amazon / Myntra SKU linked to Master SKU
- 4 New-product mapping decision flow: check if the product already exists on another connected channel — auto-map if found, else place under 'Unmapped Listings' for manual mapping
- 5 Bulk product import for new-tenant onboarding, channel-wise (Shopify, then Myntra, then Amazon)
- 6 Build Mapped / Unmapped / Failed Mapping views

Output: Product catalog with working Master SKU and multi-channel mapping

PHASE 4 — Warehouses + Inventory Engine

The single most critical phase in the entire project

Goal

Get physical vs virtual warehouse logic and overselling protection fully correct before any order or fulfilment logic depends on it.

Tasks

- 1 Warehouse table — WH-001 (Physical/Offline), WH-002 (Virtual)
- 2 Channel visibility rule: Offline stock visible on all channels; Virtual stock visible on Shopify only
- 3 Stock In / Stock Out APIs
- 4 Stock reservation logic — hold stock the moment an order is placed
- 5 Immutable inventory ledger — every stock movement permanently recorded
- 6 Overselling protection — database-level locking or queue-based handling for simultaneous orders
- 7 Low stock alerts and thresholds

Output: Inventory engine that correctly prevents overselling across all three channels

■ Do not move to Phase 5 until this phase is stress-tested with concurrent/simultaneous orders.

PHASE 5 — Orders + Fulfilment

Modules: Orders, Fulfilment

Goal

Bring orders from all channels into one Master Order and route each line item to the correct fulfilment path.

Tasks

- 1 Central order table — one Master Order per channel order
- 2 Order status tabs: New / Processing / Pending / Packed / Ready to Ship / Dispatched / Delivered / Cancelled / All Orders
- 3 Payment mode sync — COD, Prepaid, Partially Paid (treated as COD), Void
- 4 Fulfilment routing rule: line items available in Offline warehouse go to direct pack & ship; line items only in Virtual warehouse are routed to the vendor
- 5 Cancelled-then-Unfulfilled orders return to the New Orders queue (confirmed rule)
- 6 Shopify order-edit sync via webhook, reflected in the OMS immediately

Output: End-to-end order intake and correct offline/virtual routing

PHASE 6 — Vendor Management

Module: Vendors

Goal

Give vendors — many of whom are not tech-comfortable — a simple, reliable way to manage only their own assigned orders.

Tasks

- 1 Vendor login (restricted role, scoped to own data)
- 2 Assigned-orders view
- 3 Accept / Reject order actions
- 4 Simple, Excel-style entry form — minimal fields, minimal clicks
- 5 Bulk tracking update — update tracking for multiple orders shipped in one parcel, in a single action
- 6 Vendor-specific Telegram group integration — order auto-routed to that vendor's own group on assignment

Output: Working vendor panel with automated Telegram order notifications

PHASE 7 — Shipping

Module: Shipping — kept as its own phase due to third-party API complexity

Goal

Handle AWB, labels and courier integration cleanly, without mixing this third-party-API-heavy logic into Returns/RTO/NDR.

Tasks

- 1 AWB generation
- 2 Label generation, download and cancellation
- 3 Label-cancel rule: cancelling a label reverts the order to Unfulfilled and syncs that status back to Shopify
- 4 Manifest generation
- 5 Shipment tracking updates
- 6 Courier pickup request
- 7 Channel-specific courier ownership: Amazon/Myntra use their own courier; Shopify orders use Leheriya's own courier accounts

Output: Full label lifecycle with correct Shopify sync on cancellation

PHASE 8 — Returns + RTO + NDR

Modules: Returns, RTO, NDR — combined because they share the same workflow pattern

Goal

All three follow a similar request → verify → action pattern, so building them together avoids duplicated context without overloading any single phase.

Tasks

- 1 Returns: Request → Approval → Return Label → Pickup → QC → Refund/Exchange → Restock
- 2 RTO: Delivery failure → Return to Origin → Receive → QC → Restock or Reject
- 3 NDR: Failed delivery → reason capture → reattempt scheduling → escalation
- 4 Restock destination choice on return/RTO — All channels, Shopify (Offline) only, Amazon, Myntra, or None (damaged/unsellable)

Output: Complete return, RTO and failed-delivery workflows

PHASE 9 — GST Invoice + Payment & Return Reconciliation

Modules: GST Invoice, Payment Reconciliation, Returns/Refunds Reconciliation

Goal

These three share overlapping financial data and database tables, so building them together avoids rework.

Tasks

- 1 GST/HSN based invoice generation with tax calculation
- 2 Invoice PDF preview, download/print, and email delivery where supported
- 3 Amazon/Myntra payment file upload — auto-match against existing orders
- 4 Amazon/Myntra return file upload — auto-match, mark 'Return Received'
- 5 Shopify reconciliation rule: Paid/Cancelled orders auto-excluded; Fulfilled-but-Unpaid orders past a set age are flagged for review
- 6 Refund and return-driven adjustment matching, with mismatch flags

Output: Automated reconciliation replacing manual cross-checking

PHASE 10 — Notifications + Settings + Integration Management

Modules: Notifications, Settings, Integration Management

Goal

Operational/support-layer modules that configure and monitor the rest of the system — grouped since they share the same configuration-driven nature.

Tasks

- 1 Notification events — order, vendor, shipment, return, low stock — each individually switchable ON/OFF
- 2 Notification channels — Telegram, Email, WhatsApp (where supported)
- 3 Settings — company/GST details, channel connections, courier mapping, tax configuration, API credentials
- 4 Integration health dashboard — last sync status, failed jobs, webhook failures, retry actions across Shopify, Amazon, Myntra

Output: Configurable notification system and a live integration-health view

PHASE 11 — Dashboard + Reports + Audit Logs

Modules: Dashboard, Reports, Audit Logs

Goal

Build the reporting/oversight layer last, once real operational data exists to report on.

Tasks

- 1 Dashboard status tabs — New, Processing, Pending, Dispatched, Delivered orders, low stock, sales snapshot
- 2 Reports — Sales, Orders, Inventory, Vendor performance, Returns
- 3 Audit log — user, action, before/after value, timestamp for every material system action

Output: Operational visibility across the whole platform

PHASE 12 — Testing, Optimization & Deployment

Final phase before go-live

Goal

Prove the system works under real, concurrent, live conditions — not just in isolated feature tests.

Tasks

- 1 Unit, API and integration testing across all modules
- 2 Concurrency / overselling stress test — this is the real test of Phase 4's inventory engine
- 3 Role and permission testing (Super Admin vs Vendor boundaries)
- 4 Security testing
- 5 Deployment to Hostinger VPS with Node.js backend
- 6 Backup and monitoring setup
- 7 Live testing on Leheriya's actual day-to-day operations — the agreement's own completion condition

Output: Production-ready system, tested live, ready for handover

Payment Milestones Mapped to Phases

The agreement's six payment milestones (₹80,000 total) align naturally with this phase plan, so progress against payment is easy for both sides to track.

Milestone	Amount	Trigger	Corresponding Phases
Project Start	₹5,000	Before development starts	—
UI / Design Complete	₹10,000	After UI/UX design reviewed & approved	Phase 1
Backend + Database Core	₹20,000	After core backend & DB architecture complete	Phase 0, 2, 3, 4
API / Channel Integration	₹15,000	After Shopify, Amazon, Myntra integration complete	Phase 3–7 (channel APIs)
Business Workflow Completion	₹15,000	After inventory, vendor, fulfilment, shipping workflow complete	Phase 5, 6, 7, 8
Final Testing & Delivery	₹15,000	After final live testing, code + DB handover	Phase 9, 10, 11, 12
TOTAL	₹80,000		

Reference

This roadmap is a development-sequencing guide built on top of the Final Combined Agreement & Scope (Leheriya Creations × Momtaj Husen, ₹80,000, 23 modules). It does not alter any commercial term, module scope, or business rule in that agreement — it only sequences the agreed 23 modules into build-safe phases for use with Cursor AI.